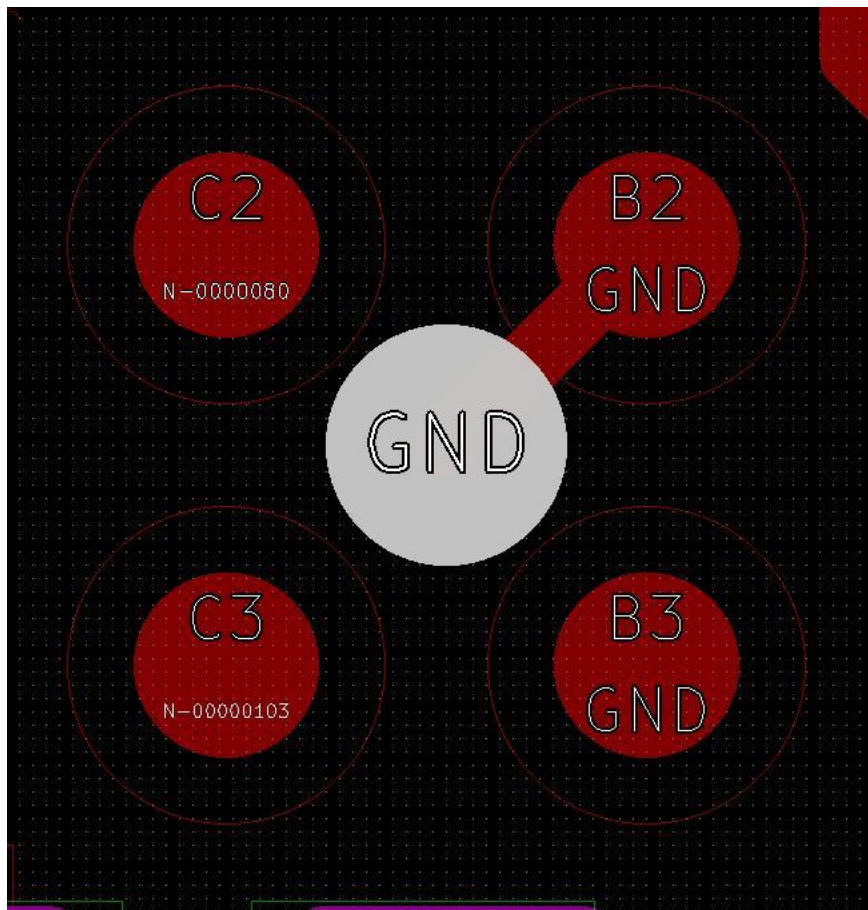


制作自己的嵌入式 Linux 电脑

当今所有最好的集成电路都大规模的使用 BGA 封装法来焊接。因为 BGA 封装法连接在芯片底下，焊接更紧，需要使用回流焊箱或者热印版。另一个问题是 设计 PCB（印刷电路板）时，过孔和引线之间的焊接球需要足够小，主板上通常需要更多层来为紧挨的引线来腾出空间，这意味着一个廉价的中国产的两层主板没有足够的空间，所以需要更多的层。附加层则会显著提高主板的成本，就算只多了几个拷贝。

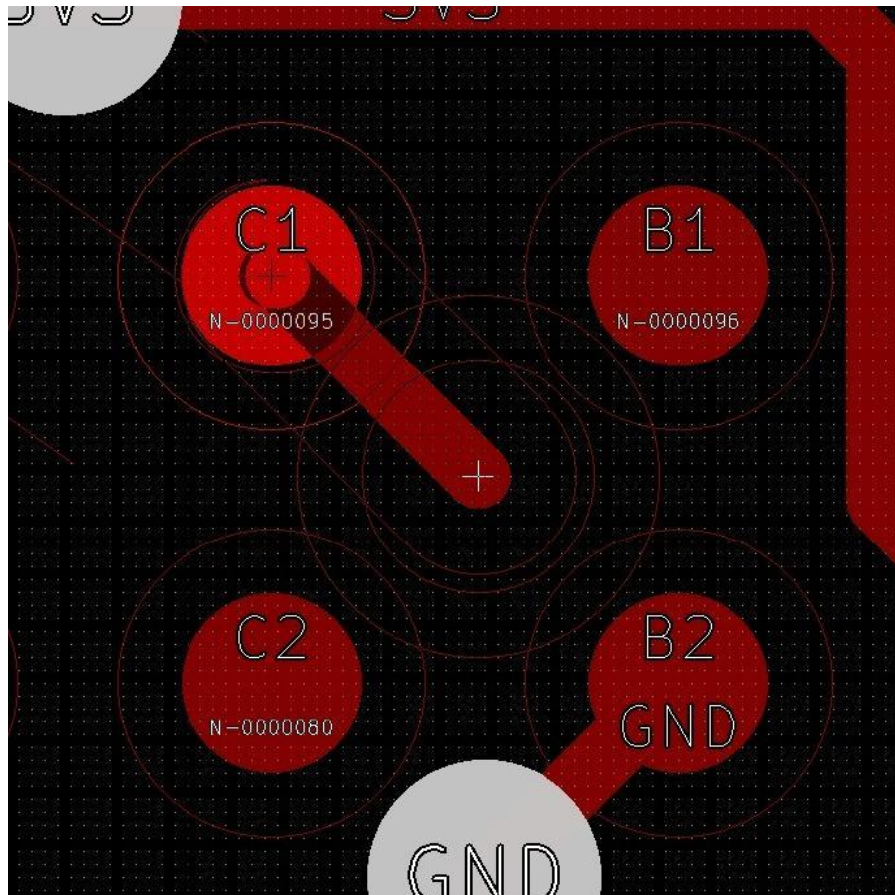
我想设计一款内置 BGA 芯片的主板来体验下焊接它们究竟是有多难。于是我决定设计一个可运行 Linux 的小型 ARM 嵌入式系统，使用的 ARM 处理器 是在一个 217 球的 LFBGA 包中的 AT91SAM9N12，只是因为带有运行 Linux 必需的内存管理单元的 ARM 处理器中，它是最便宜的。起初我只 想用一块 BGA 芯片，但是 BGA 包中的 RAM 比其他包里要便宜很多，所以我就决定在 BGA 包也增加一块 DDR2（Double Data Rate 2）的内存。



为最大化可用空间定位过孔。

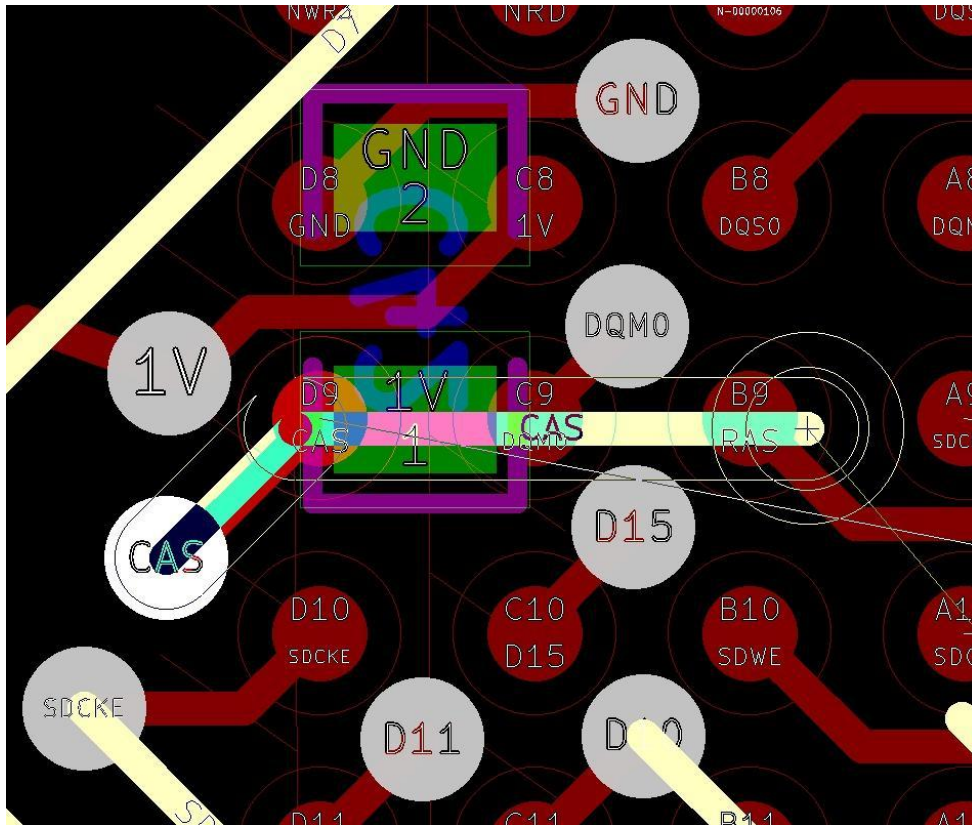
结果寻找主板生产商还颇费了一番周折。两层的空间是还不够的，至少需要 4 层。217-LFBGA 包的球直径是 0.4mm，临接球的距离是 0.8mm。为了给过孔多留些空间，球的焊盘布局做的要比焊球要稍小一些。我用的 0.36mm 的焊盘。在 4

个球之间放置过孔会最大程度的利用可用空间。厂 商要能制造可以放进 0.8mm 宽度的过孔。几乎所有厂商都可以制造这种直径大小的过孔，但问题是：这个距离包括了过孔的钻孔直径，两倍的过孔绕环的宽度，两倍的过孔和引线之间的最小距离。比如，iTead 的 4 层主板最小的过孔钻孔直径是 0.3mm，最小环宽度是 0.15mm，过孔和引线的最小距离是 0.15mm，加起来是 0.9 mm，这意味着最小尺寸的过孔不能放在 BGA 球之间。我发现的唯一一家可以实现这一要求而且价钱相对合理的生厂商是 OSH parks。他们的四层主板有更小的限制，过孔刚刚可以放进 BGA 球里面。额外的好处是，对于小主板而言，它要比 iTead 更便宜一些。



在 OSH park 设计原则下最小的过孔，刚刚能放下。

即使过孔可以放进 BGA 球中间，仍然有一些问题：过孔中间没有足够的空间走线。这意味着要让每一个焊盘都有一个过孔的标准布线通道是不可能了。这就是说主板需要有足够的未经连接的焊盘，所以过线需要从里面进行布置。幸运的是，处理器还有很多通用的未连接的 I/O 引脚。



… 如果不违反设计原则的话，过线就不能正好在两个过孔之间穿过。CAS 过线没有足够的空间来放入 DQM0 和 D15 过孔。

生产问题解决了，是时候开始想想主板上该放哪些部件了。我并不很在乎这块主板的实际用处，相对于用处而言，整个项目更是一个学习的过程。为了降低成本，主板的尺寸要小。这意味着不会为其他额外的接口预留空间，比如：以太网，串口或者 SD 卡。

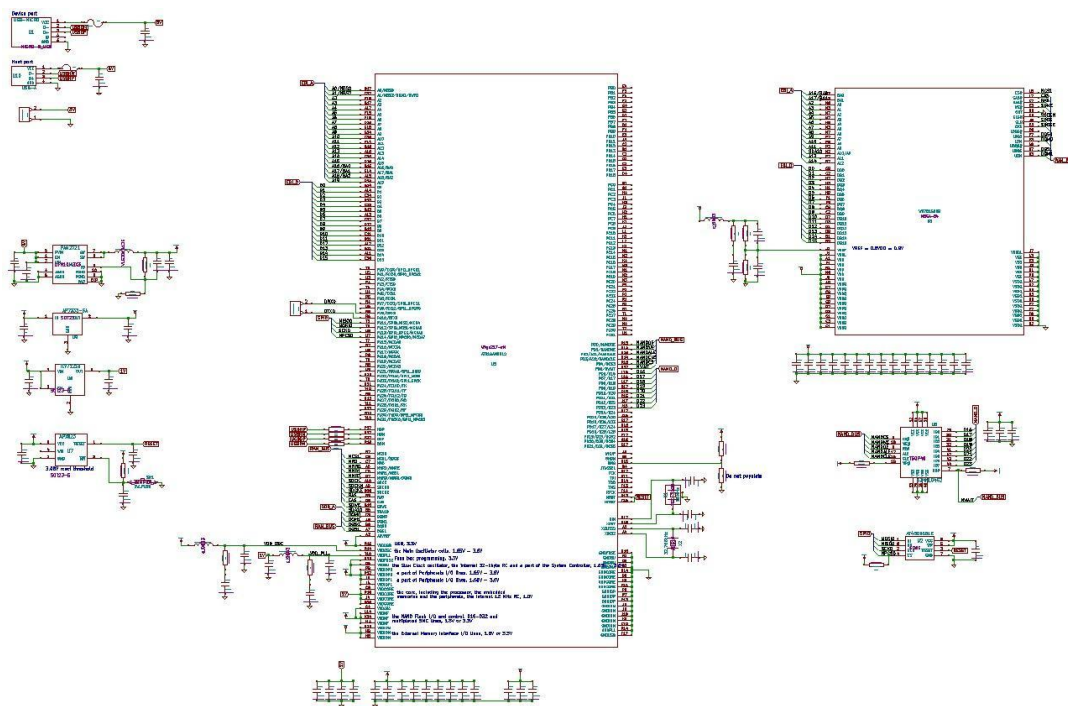
除了处理器和 RAM 外，其他必需的部件是：大内存，电压调整器，以及处理芯片重置的监控电路。处理器可以从 NAND 启动，但是以防万一我决定为引导 装载程序加入 Dataflash（数据闪存），虽然最终会很少被用到。对于大内存而言，NAND 是一个很好的选择因为他容量大又便宜。在 BGA 包中加入会 更便宜些，但我已经被两个 BGA 包折腾的够呛了，所以我决定在一个 48 引脚的 TSOP（薄型小尺寸封装）包里面使用 4GB 的 NAND。连接各个组件在处理器清单表中已经解释的很好了，但是上千页的文档中找到全部的细节还是很难的。Atmel 也发布了一个试用板的原理图，在设计主板时会很有帮助。

DDR2 引线空间应该有一定的自由度。正常的引线应该长度合适，有可控的阻抗和可以终止或者串联电阻。在开发板的参考设计中，所有 DDR2 的信号使用了串联电阻。我没有足够的空间放置他们，所以我决定暂且放着不管。阻抗也不是 50 欧姆，因为我必须使用小一些的引线来填充其他的空间。我希望的是，因为 RAM 更靠近处理器，就算缺少串联电阻箱或者阻抗不匹配，关系也不大。所有从 CPU 到 RAM 的连线大约是 25mm 长。通常的经验是：如果引线的长度要超过信号波长的 10% 时，转换线的影响应该被考虑进去。这种情况意味着频率大约在 1

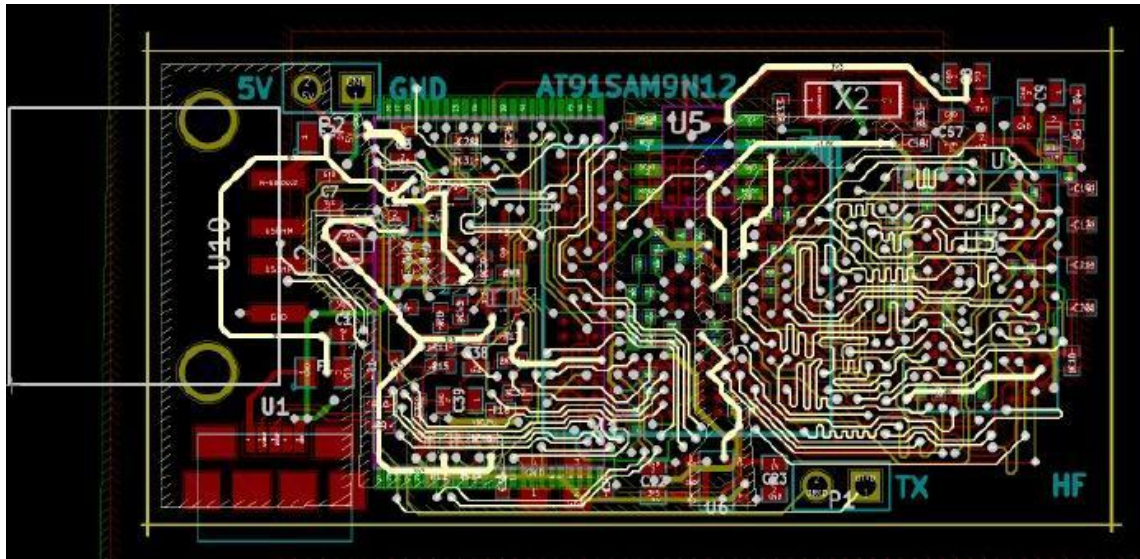
GHz 以上。RAM 的时钟频率只有 133 MHz，甚至头几个谐波还在 1 GHz 以下，这预示着应该会正常工作。为了保证可行，我几乎完全匹配了引线的长度，但这也许不是必须的。

供电有些复杂。处理器核心的电压是 1 伏特，RAM 需要 1.8 伏特，NAND 需要 3.3 伏特。因为从 USB 输入电压是 5 伏特，主板需要有三种不同的电压适配器。正常情况下比较好的做法是：在主板上为电力供应保留一层并且保持它与信号脱离，来降低电力供应的阻抗，但是主板只有 4 层，而且其中一层要用于做底板。这意味着只有两层留给做信号处理，这显示不够。所以我没有单独拿出一层来供电，而是在不同的层里为不同的供电做了一些挡板。

对于 USB 供电的应用，线形调整器的电力损失在最坏的情况下太大了些，所以我决定用 3.3 伏特的调整器作为一个更有效的可切换开关的调整器。1.0 伏特和 1.8 伏特的调整期是一个以 3.3 伏特作为输入电压的线性调整器。因为线形调整器的损失决定于输入和输出电压的差，所以使用 3.3 伏特的电压比 5 伏特的电压提高了效率。



电路图。PDF

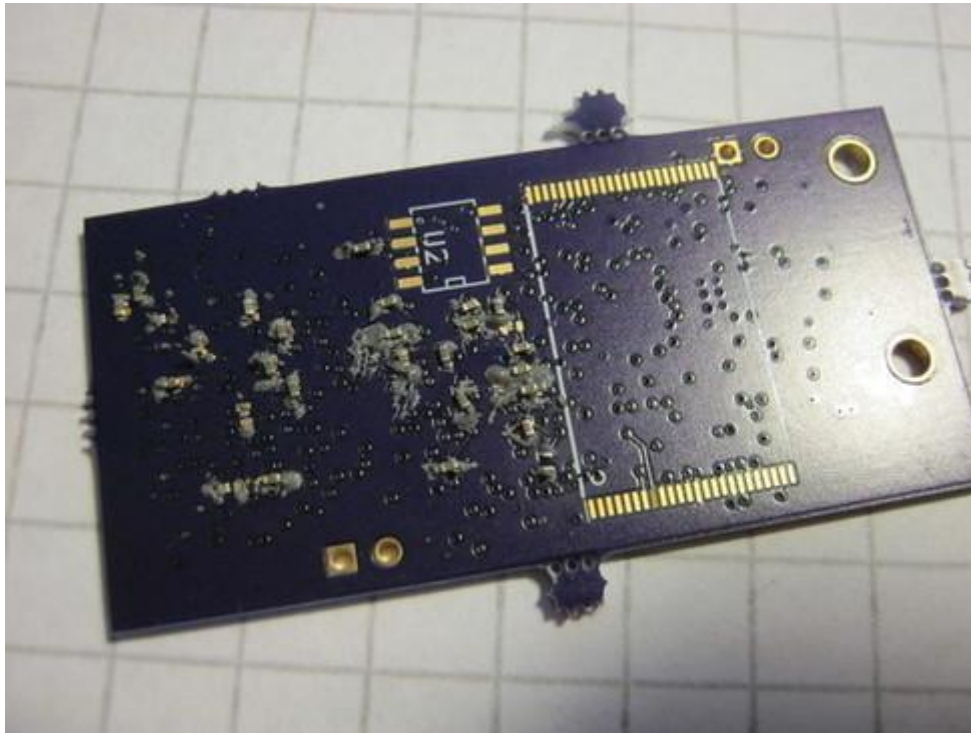


PCB 布局，尚未焊锡。

焊接



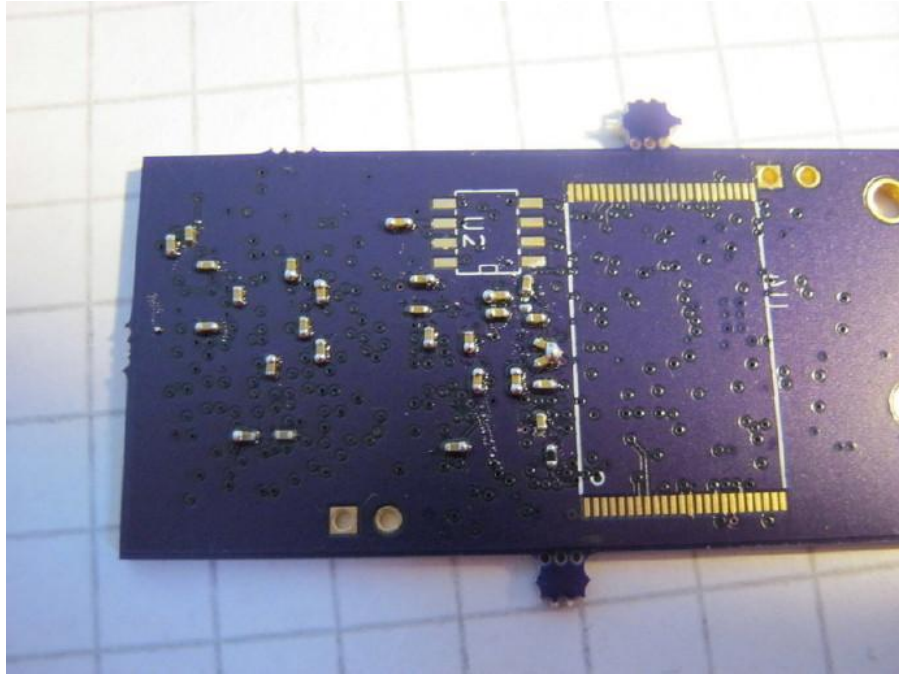
空的电路板。



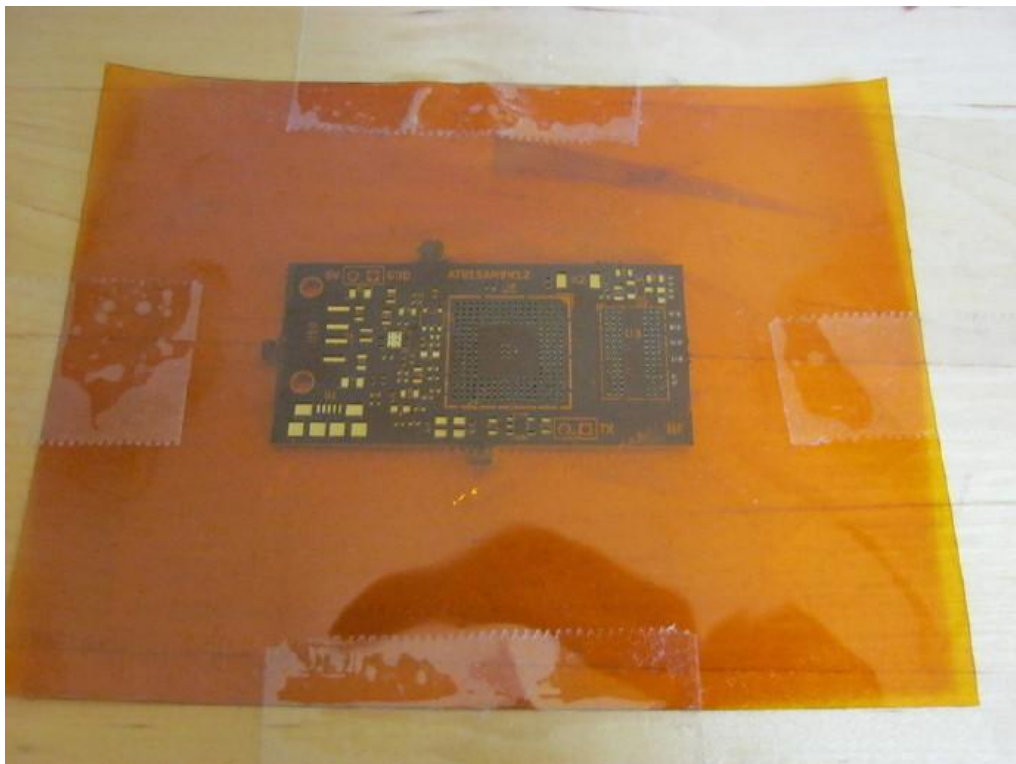
焊锡和部件被固定到背面。焊锡是用牙签手动固定的。这儿的部件都是 1mm (0.04 英寸) 长。我只拿上一些做下试验，先看下他们如何回流的。如果情况不太好，我会换另外一块板子。



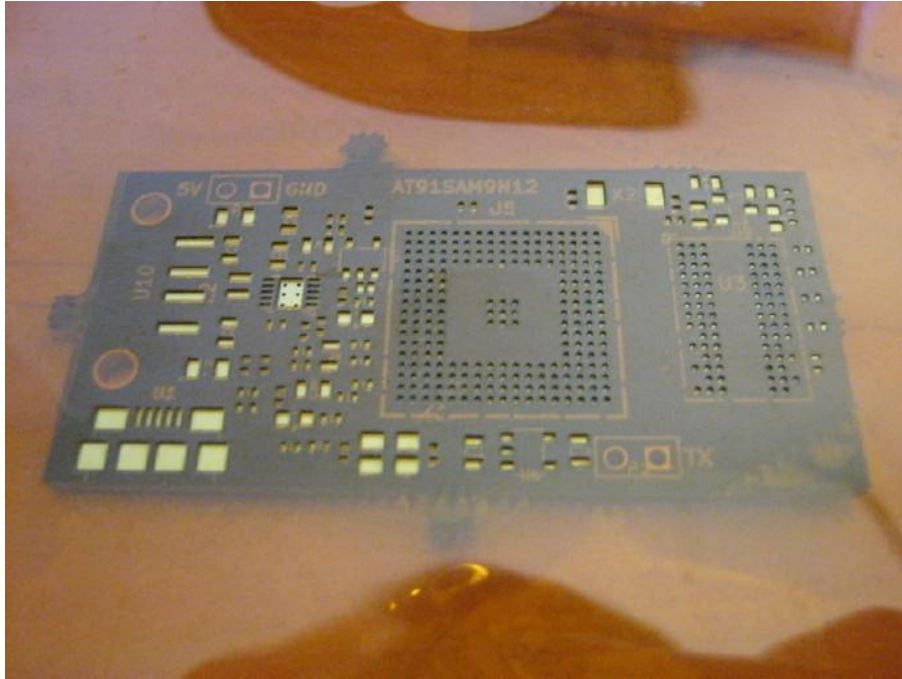
使用一个烤箱和自制的控制器来控制回流。



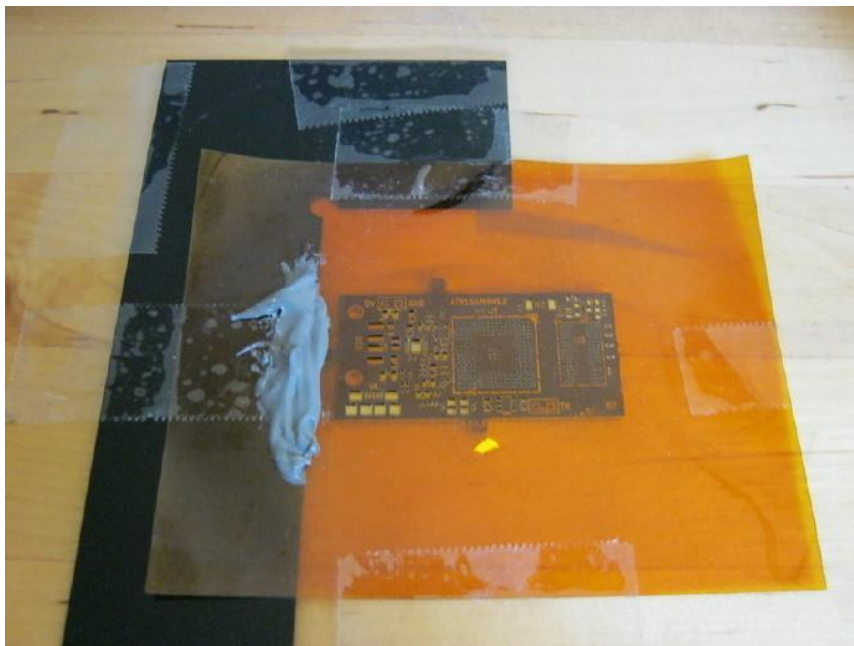
回流之后。三个部件被焊接到一个错误的地方。我最后还是把它们拿出来了，在主板上有足够的解耦的电容器，即使丢了一小部分，也不过有什么影响。我还错误的把一个电容器放在左上的位置，但那儿其实应该是一个电阻。



对于上层，我有一个 OSH 的模板，所以我不必手动把焊锡弄到 BGA 板上。直接按到桌子上，我就把主板和模具焊牢了。



模板队形排列很整齐。



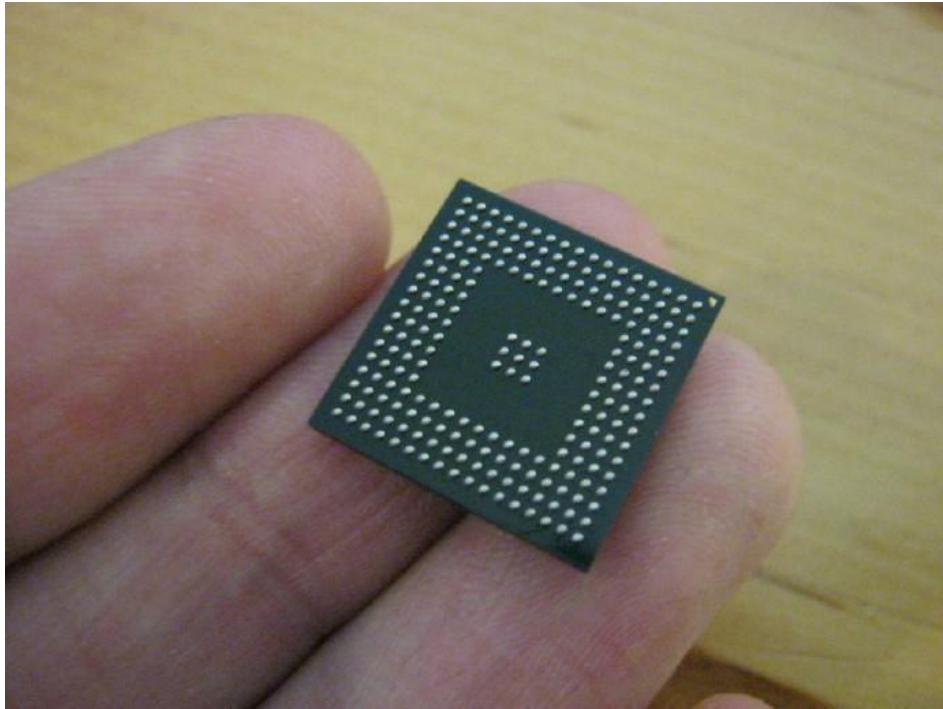
这看起来有些过了，但是几乎所有的焊锡都可以用了。还需要一些额外的焊锡来让模具平整一些。



把焊锡铺开，然后挪掉模具。比我自己手动弄的背面要好的多。



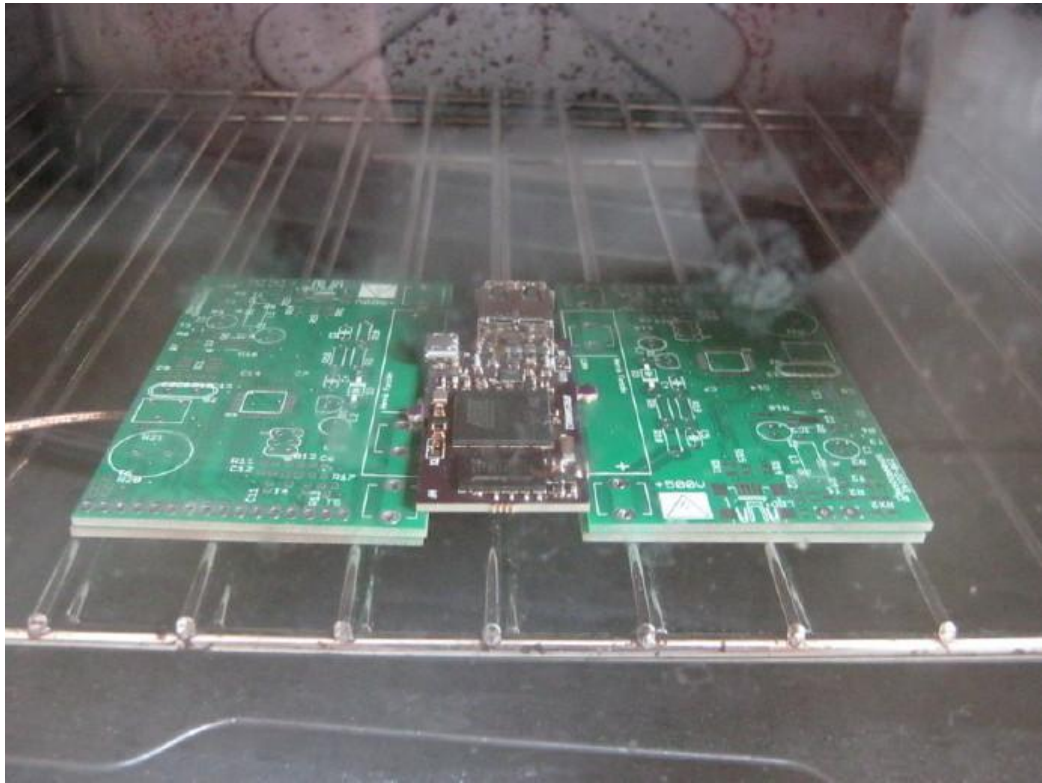
我从一个非 BGA 的部件开始。它们是用一双沉稳的手用钳子固定的。



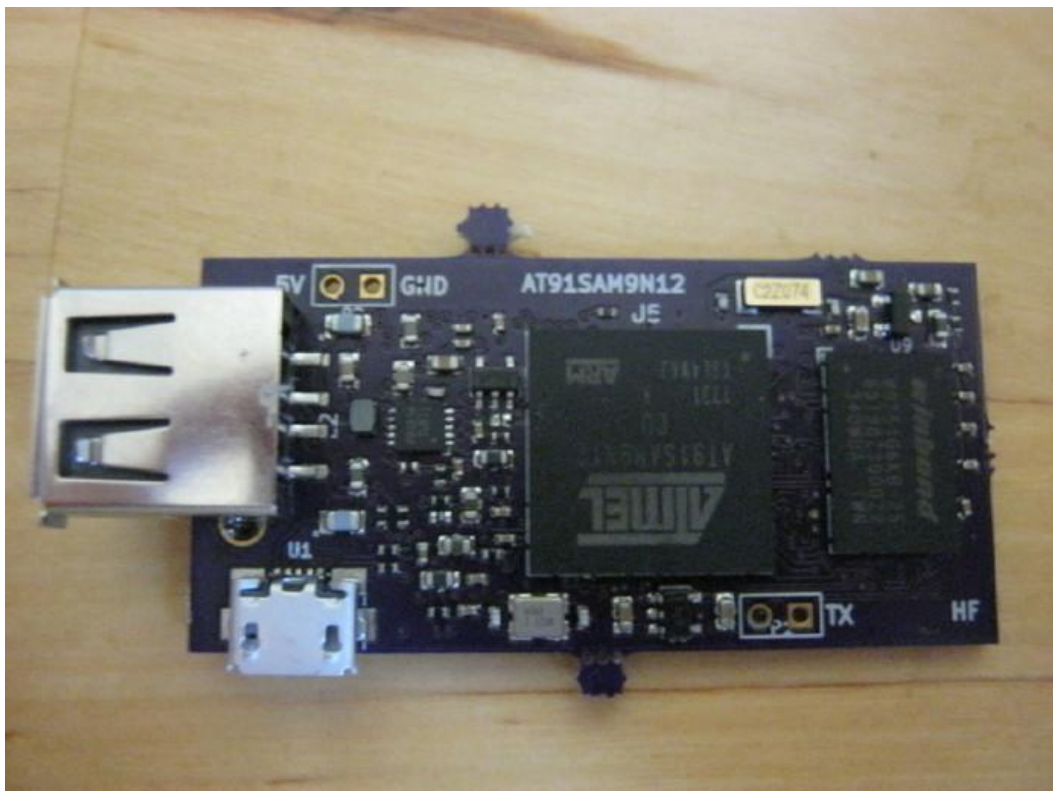
CPU 和我的指尖。球间距是 0.8mm。许多新的 BGA 甚至使用小于 0.5mm 的间距。



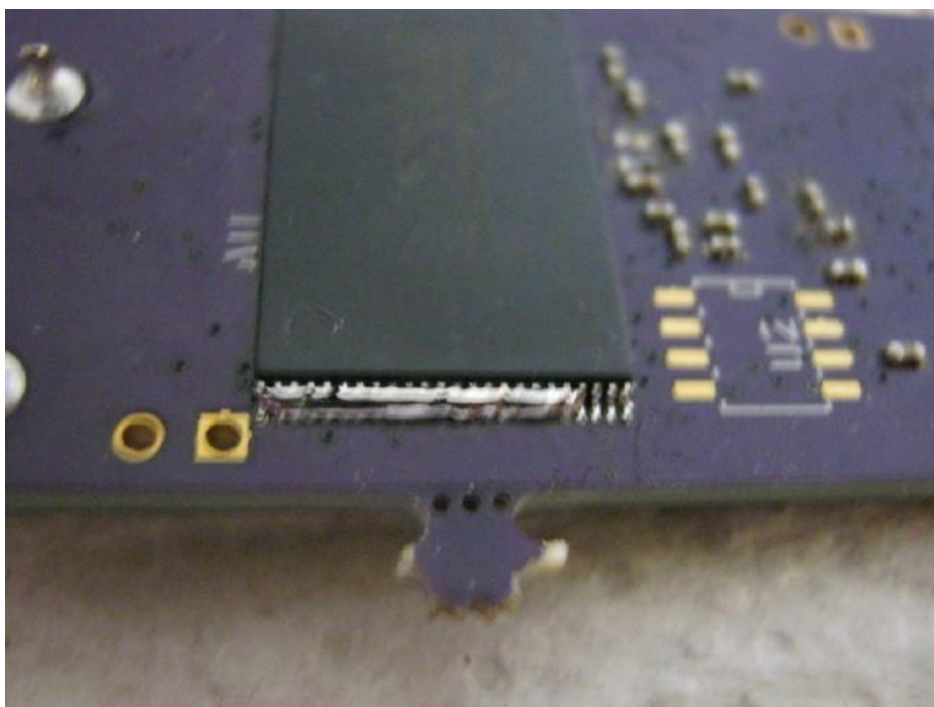
BGA 固定在板上。部件放置的位置误差需要小于 0.4mm，否则可能和一个行间隔焊接，而且因为焊球在芯片下面，不能检查。没有丝网印刷的边界，几乎不可能按照要求的精度放置。有了丝网印刷，只需沿着丝网印刷边沿对齐即可，很容易。



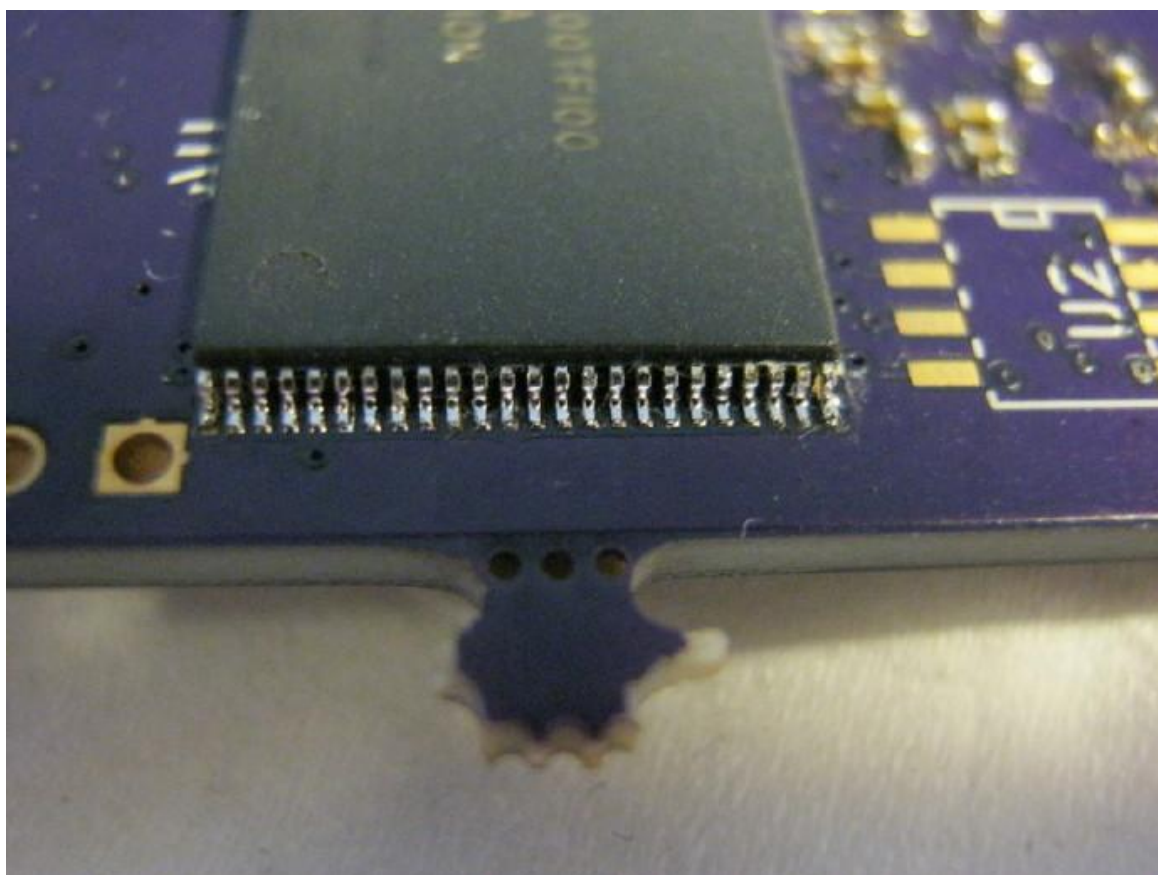
回焊正面，抬高 PCB，这样底面的部件也不会触到其他地方。焊接表面的拉力会保持底面不倒下。



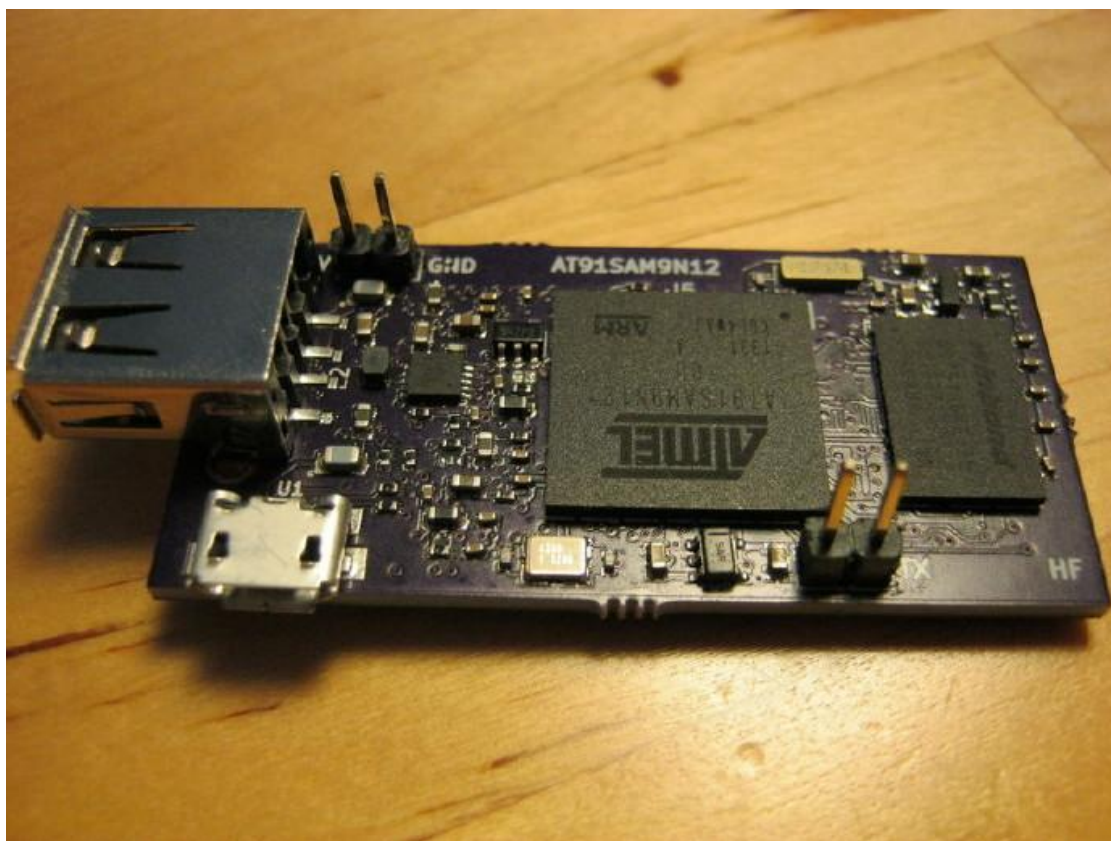
过烤箱后。焊接口看上去很好看，所有部件仍然在他们应该的位置上。



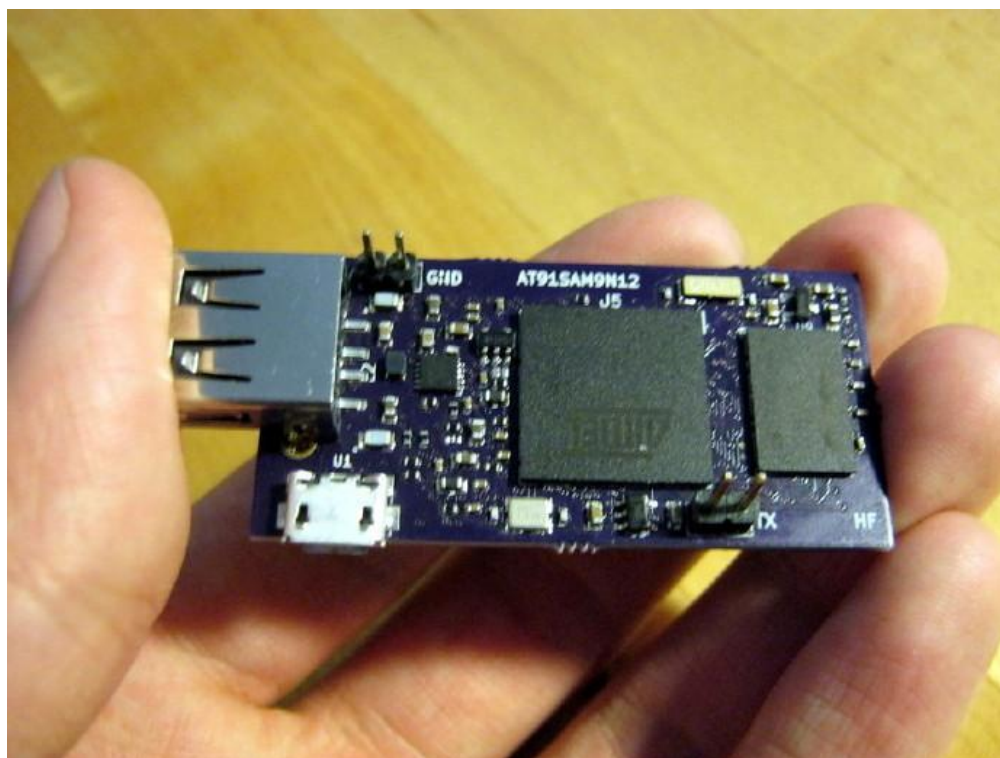
焊接 NAND Flash。我的焊接铁片要比引脚大一些。一次焊接一个引脚太困难了。简单些的办法是灌锡后，用吸锡带把多余的吸出来。



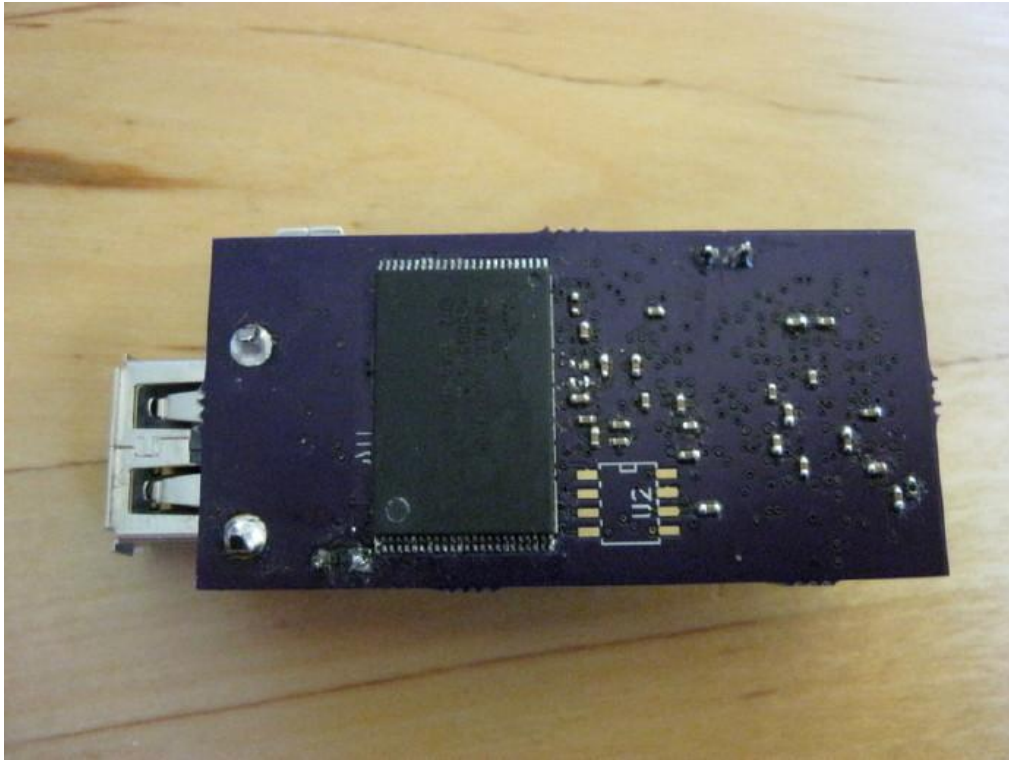
在移除多余焊锡后，焊接口现在质量很高了。



在加入供电头和调试串口后，主板焊接完毕。

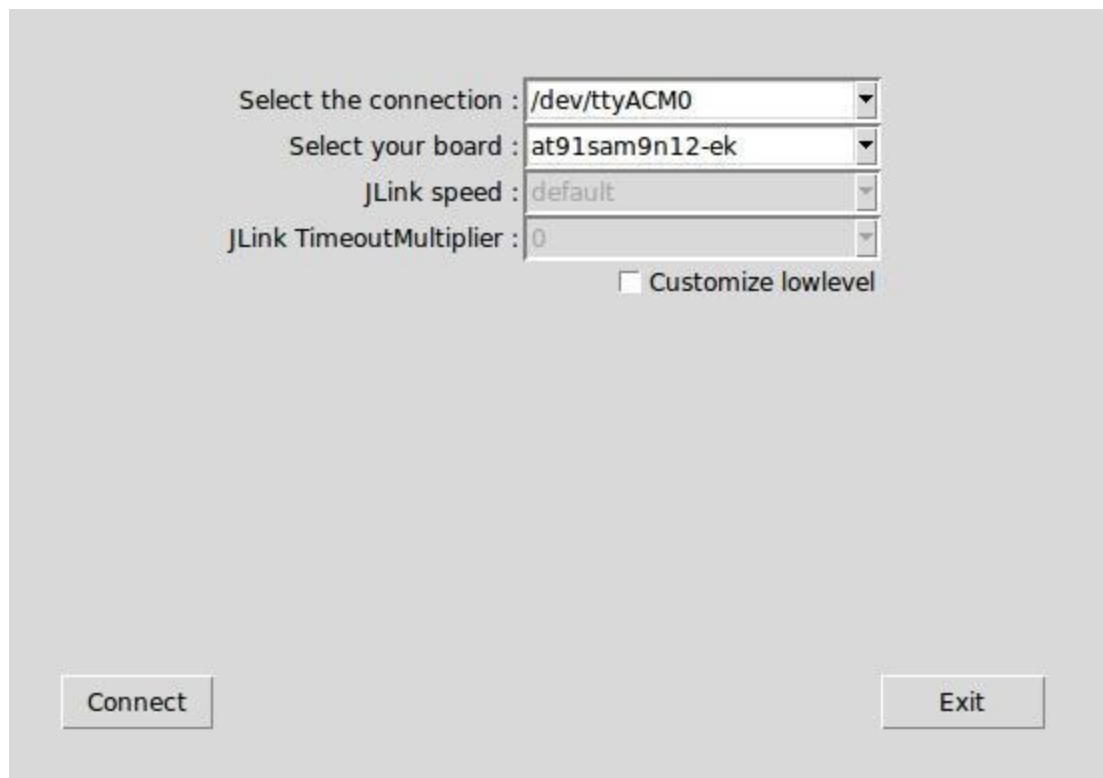


最终的成品。

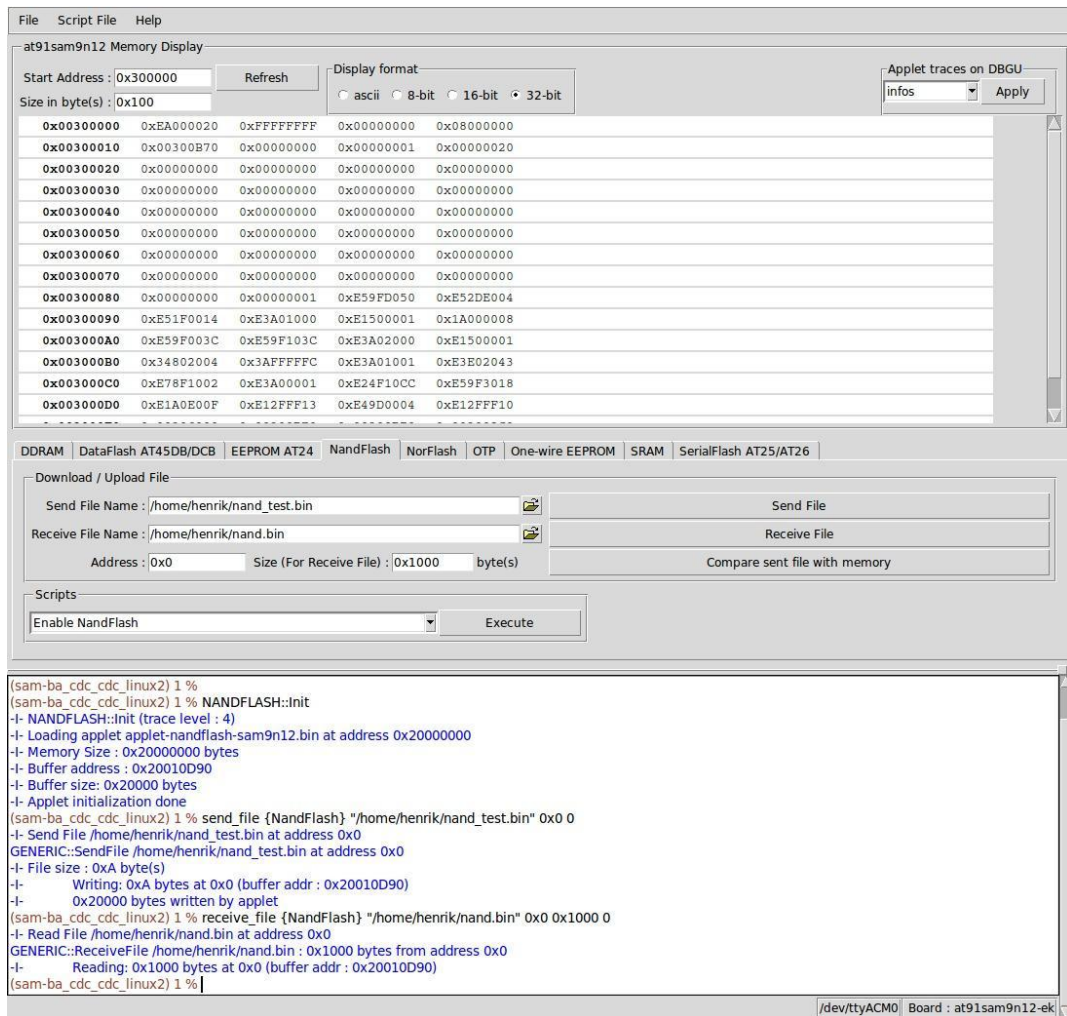


另一面，空着的位置是位 Dataflash 准备的。

在把 USB 缆线接到 USB 设备口上，没有出现什么意外，而且我看到了一个新的串口 `/dev/ttyACM0` 出现。再用 SAM-BA 程序（用来对启动加载程序和内核编程）打开，一切都能用了。许多人说焊接 BGA 很难，但是从这次经验来看，我觉得还好。也许只是幸运吧，但是我的确没出什么问题就搞定了。

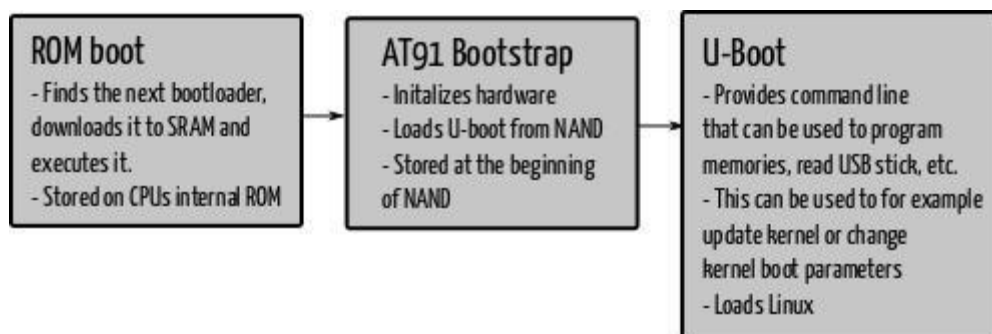


打开 SAM-BA。At91sam9n12ek 是 Atmel 针对这款处理器提供的开发工具，他的配置同样适用于这块板。



DDR2 也可以工作，执行程序，并且可以写回 NAND。一切都 OK。

软件



软件启动加载程序

启动进程开始工作时，内部的 ROM 加载器尝试在不同的内存区域发现一个合法的程序。他会扫描 SPI（串行外设接口）缓存，SD 卡，NAND flash，二级 SPI flash 还有 I2C EEPROM 对于一个合法的程序。一旦发现一个，马上就启动它，否则就

会进入 SAM-BA 监控器，也就是进入调试模式，此时处理器会监听来自串口和 USB 端口的指令。这种模式下可以对 bootloader 进行编程。

ROM 启动不能直接启动 linux 内核，所以需要有一个二级的启动加载器。它会初始化 RAM 和时钟，然后开始加载 linux 内核。AT91 Bootstrap 是一个现成的启动加载器，可以完成这一系列操作。它被放在 NAND flash 开始的地方，或者如果我填充这些位置，它可能被放到 Dataflash。即使 AT91 bootstrap 可以直接启动 linux，对于调试而言，之后启动 U-boot 的 bootloader 对调试更有用。U-boot 是它自带的基于命令行的微操作系统，可以读取 USB 棒，使用以太网，读写 NAND，当然可以启动 linux。例如使用 U-boot 可以清除 NAND 或者改变 linux 启动参数。

为编译 bootloader，需要一个 ARM 交叉编译器。我用的是 Sourcery codebench lite edition，因为它容易设置，而且效果不错。先加载 AT91SAM9N12EK 开发板配置文件是最容易的。和从头开始写新的配置相比，修改文件要省事的多。

为了让这份定制版同样有效，需要做一些改动：RAM 大小需要设置成 64MB，bank 的数量改成 4，一些试验需要稍微调整一下（试用版有 128MB 的 RAM 容量，8 个 banks）。NAND 初始的函数也需要改动，这块板比开发板相比，NAND flash 要连接的地方有些不同，因此有必要告诉 bootloader。

U-boot

因为 AT91 bootstrap 已经初始化了整个硬件，就可以直截了当的配置 U-boot。同样有为 at91sam9n12ek 准备的配置文件，但是默认是从 SD 卡启动设置的。因为硬件已经配置好了，所以其他需要改动的地方就没多少了。有一些可选项比如在 NAND flash 上创造和改变分区时启动 UBIFS 工具，以及为读取 ext4 格式的 USB 棒提供支持。USB 支持使得从 USB 棒启动 Linux 内核变得可能，这让试验不同的内核配置变得容易了。

Linux 和根文件系统

安装 linux 不像安装一台常规的 x86 pc 一般容易。需要配置内核来支持各种需要的设置，而且需要建立根目录的镜像文件。可以手动来做，但是用 buildroot 做起来会很容易些。后者是用来建立根文件系统和内核的一系列工具。整个过程可能会有一些难，因为内核和 build root 有很多的选项。

Buildroot 没有为 at91sam9n12ek 开发板提供配置文件，但有为另一款 Atmel 板 at91sam9260ek 提供配置文件。使用这个配置文件作为基准文件，配置会更容易些。可以通过“make at91sam9260ek_defconfig”加载。

开始我们想有一个相对新的内核版本，因为相对于处理器来说，只需要做一些很小的改动，所以我们就使用上周新发布的 3.15.1 版本吧。

Linux 被配置成用 buildroot 的 “make linux-menuconfig” 命令配置。它会打开寻常的 Linux 菜单配置窗口。内核中大多数重要的配置是系统类型的配置菜单。我们需要检查 AT91SAM9N12 的支持情况还有 “Atmel AT91SAM Evaluation Kits with device-tree support” 选项。设备树是一个随内核一同加载的外部二进制文件，描述了开发板上各硬件的可用性情况。这使得使用带有不同开发板的同样的内核以及针对不同开发板的设备配置，可以用文本来写，而不是为每块板都写一个只是略有不同的 C 文件。幸运的是：有一个可以作为基准文件的针对 at91sam9n12ek 的设备树文件。需要做得改动只是一些简单的移除不再使用的设备。这块修改过的设备数文件需要被添加到 buildroot 配置，所以它可以知道如何编译，并且从中构造一个镜像文件。

内核里面其他值得启动的选项是：

USB 主机支持 AT91_USB。

支持 NAND flash 还有处理器内部的 NAND ECC 控制器支持。

支持 UBIFS，将被用作一个根文件系统。

读取 USB 棒的 Ext4 支持。

在 buildroot 配置过程中，我们需要选择我们想要在根目录上安装哪些程序以及产生根文件系统镜像的选项。这块开发板有不带控制器的原始 NAND 内存，所以一般的桌面文件系统，比如 ext4 就不能用了。UBIFS 是通常的选择，在这正好也能用。

UBIFS 有一些依赖 NAND flash 类型的选项，如果设置错了，Linux 将不能读取最终的文件系统。这些选项可以从 NAND FLASH 的数据清单里面得到，但是更容易的方法是从 USB 棒启动 linux，并且从那创建 ubi 分区。或者也可以使用 U-boot 的 “ubi info” 指令，这将会读取 NAND 并且输出需要的配置值。

在键入 “make” 后，Buildroot 会下载交叉编译器，linux 内核以及所有其他的包，构建并输出内核，设备树，以及根文件系统镜像。然后可以用 SAM-BA 程序传到开发板上。一些程序需要对 NAND ECC controller 参数编程。一些参数也应该被配置到 AT91bootstrap，U-boot 以及 linux 内核，否则他们会报告 NAND 已经损坏。这种情况下 NAND 有 2048 个字节，带有 512 字节的扇区，ECC 能够每 1 个扇区纠正 4 个字节。存储这些镜像的 NAND 地址可以在 AT91 的 bootstrap 以及 U-boot 配置文件中找到。

在对主板编程以及重置后，ROM 启动加载器应该可以在 NAND 上找到 AT91bootstrap 并且开始启动过程。如果你对更多细节感兴趣，所有硬件和软件的文件都可以在这找到。